

把下載資料搬進 Linux 課程資料夾

這一步只做路徑準備：把 release 帶進 Linux，再確認程式根目錄

① 這個工作區

WSL = Windows 內的 Linux
讀
`/mnt/c/.../Downloads`

② 程式不改

`student_policy.py` 不動
只複製 release 副本

③ 進入根目錄

下一步集中到
Linux
`lora-energy-lab/`

④ 核對結果

執行 `pwd` 只讀位置
路徑不對就先停止

先把下載副本放進 Linux，再確認根目錄；路徑錯會讓後續設定讀錯檔

`pwd`（目前目錄命令） | 來源：Linux / macOS 終端機（WSL 也使用 Linux 指令） | 作用：確認目前位置
單位：路徑文字 | 判讀：輸出應位於 Linux home 的課程資料夾

先找到或安裝 Python 3.11

這一步只找符合版本的 Python；找不到或版本不符就停止

Linux / WSL 先檢查

```
command -v python3.11
python3.11 --version
找不到：sudo apt
install -y curl
再 uv python install
3.11
```

macOS 先檢查

```
command -v python3.11
python3.11 --version
沒有時：brew install
python@3.11
```

Windows PowerShell

```
py -3.11 --version
找不到：winget install
--id=astral-sh.uv -e
再 uv python install
3.11
```

Windows Command Prompt

```
py -3.11 --version
只讀版本結果
沒有 3.11：回
PowerShell 安裝
```

先檢查 → 缺少就安裝 uv / Python 3.11 → 再檢查；版本不對就停在這一頁

PYTHON_BIN（Python 3.11 執行器） | 來源：README.zh-TW.md | 作用：選擇符合版本的 interpreter

單位：path | 判讀：結果必須是 Python 3.11.x；不符合就停止，不建 venv

確認 venv 可用並建立 .venv

這一步先測 venv module，再建立隔離資料夾；失敗就停，不安裝套件

Linux / WSL

```
"$PYTHON_BIN" -m
  venv --help
"$PYTHON_BIN" -m
  venv .venv
成功：出現 .venv/
```

macOS

```
"$PYTHON_BIN" -m
  venv --help
"$PYTHON_BIN" -m
  venv .venv
成功：出現 .venv/
```

Windows PowerShell

```
& $python311 -m venv --
  help
& $python311 -m
  venv .venv
成功：出現 .venv\
```

Windows Command Prompt

```
py -3.11 -m venv --help
py -3.11 -m venv .venv
成功：出現 .venv\
```

先測 venv module → 建立 .venv；出現 venv / ensurepip 錯誤就停，不執行 pip install

.venv（隔離 Python 環境） | 來源：Python 3.11 venv module | 作用：建立 package-local 環境
單位：directory | 判讀：看到 .venv/ 或 .venv\ 才進入啟用；失敗回到 Python 版本頁

啟用 .venv 並確認 Python 路徑

這一步讓 `python` 指向隔離環境；版本或路徑不對就停

Linux / WSL

```
source .venv/bin/activate
python --version
python -c 'import sys;
assert sys.prefix !=
sys.base_prefix;
print(sys.executable)'
```

macOS

```
source .venv/bin/activate
python --version
python -c 'import sys;
assert sys.prefix !=
sys.base_prefix;
print(sys.executable)'
```

Windows PowerShell

```
.\.venv\Scripts\
Activate.ps1
python --version
python -c "import sys;
assert sys.prefix !=
sys.base_prefix;
print(sys.executable)"
```

Windows Command Prompt

```
call .venv\Scripts\
activate.bat
python --version
python -c "import sys;
assert sys.prefix !=
sys.base_prefix;
print(sys.executable)"
```

啟用 → `python` 指向 `.venv` → 版本與 `sys.prefix` 通過；不通過就停在啟用頁

`sys.prefix`（隔離環境判斷） | 來源：Python runtime | 作用：確認目前 shell 已啟用 `.venv`
單位：path | 判讀：`sys.prefix` 不等於 `sys.base_prefix`，且執行檔位於 `.venv`；否則不安裝

在 venv 安裝鎖定套件並驗證 READY

四種 shell 都先安裝鎖定檔再驗證；看到 **READY** 才進案例，失敗就停

Linux / WSL

```
python -m pip install
--require-hashes --no-
deps
-r requirements-lock.txt
bash course.sh verify
```

macOS

```
python -m pip install
--require-hashes --no-
deps
-r requirements-lock.txt
bash course.sh verify
```

Windows PowerShell

```
python -m pip install
--require-hashes --no-
deps
-r .\requirements-lock.txt
.\course.cmd verify
```

Windows Command Prompt

```
python -m pip install
--require-hashes --no-
deps
-r requirements-lock.txt
course.cmd verify
```

locked requirements → verify → status: READY；沒有 **READY** 就停止，不執行 lab

verify-receipt.json（驗證收據檔） | 來源：*course.sh* / *course.cmd verify* | 作用：記錄 Python、scenario 與 policy contract gate
單位：JSON artifact | 判讀：看到 *status: READY* 才能進案例；它不是 *result.json*

卡住時修復或保留來源

這一步處理卡關：修得好就重跑，修不好也不冒充本機執行

① 正常

READY 後執行案例
產生新結果

② 修復什麼

只修
Python、root、lock
不改 policy block

③ 回復什麼

同情境結果成對保存
result.json / *endpoint-replay.json*

④ 看什麼

讀 *artifact_source*
判斷本機或回復

修復不改策略；回復不冒充本機執行

artifact_source（資料來源分類） | 來源：結果紀錄 | 作用：標示本機執行或回復資料
單位：來源值 | 判讀：來源模式保持原值，不改寫成本機執行

只可修改三個標記區塊

這一步畫出可編輯邊界：三個實驗區可改，其他程式只讀

① 唯一檔案

只在這裡做實驗
`student_policy.py`

② 可改區段

A / B / C 三個標
記
marked block

③ 保持不變

中文註解、函式、情
境
runner、schemas 不
動

④ 下一步／證據

只開目前 lab
結果仍看 result /
replay

只開目前實驗區；其他檔案與輸出契約保持不變

marked block（標記區段） | 來源：檔案標記 | 作用：界定一個實驗的可編輯範圍
單位：程式碼區段 | 判讀：其他套件邊界維持唯讀

程式每次判斷會看什麼

這一步先說清楚輸入：每個 step 只看目前／過去資料

① 讀目前資料

窗口、急件、間隔、
品質
都是當下 snapshot

② 不讀未來

不看結果摘要
不回看 energy

③ 交給函式

把目前資料交給
`choose_action()`
再回一個合法動作

④ 下一步／證據

回一個合法動作
runner 再記 event /
result

當下觀察 → 合法動作；這一頁沒有程式值變更

`choose_action`（策略函式） | 來源：student_policy.py | 作用：依當下觀察回傳一個合法動作
單位：函式契約 | 判讀：只讀目前／過去欄位，不讀未來結果

判斷順序如何產生動作

這一步把合法動作順序翻成白話；鏡像可讀，不能貼回程式

① 窗口關閉

不能送出，先休息
SLEEP

② 期限逼近

急件先送出
SEND_URGENT

③ 節奏／品質

間隔未到休息；品質未
穩等待
REST_DURING_GAP /
WAIT

④ 佇列／保底

夠量批次，否則單筆；
無事等待
FLUSH_BATCH /
SEND_ONE / WAIT

先看窗口，再看期限、間隔、品質與佇列；順序會改變動作

action（合法動作） | 來源：policy API | 作用：把條件轉成 runner 可執行的動作
單位：enum | 判讀：下一步從 state / packet / service / endpoint J 讀後果

Lab A：空檔休息與等待

Lab A 的目的：比較空檔休息與清醒等待，找服務／能量取捨

① 檔案／區段

student_policy.py
lab-a-pace-rest

② Before (唯一變更)

空檔休息
SLEEP

③ After (唯一變更)

清醒等待
WAIT

④ 固定／觀察

gap=2；A/B/
C、runner 不動
看 service / state / J

只改空檔動作 → 改變休息／清醒／喚醒 → 影響服務與端點焦耳

REST_DURING_GAP (空檔動作) | 來源：Lab A marked block | 作用：控制送出間隔中的狀態
單位：action | 判讀：比較 *SLEEP* 與 *WAIT* 對狀態、服務、端點 J 的影響

Lab C：批次與期限餘裕

Lab C 的目的：比較批次門檻與期限餘裕，找傳送時機代價

① 檔案／區段

student_policy.py
lab-c-batch-urgent

② Before (唯一變更)

期限餘裕
20 秒

③ Candidate / Revision

候選 5 秒
修訂 30 秒

④ 固定／觀察

BATCH_SIZE=3 ; A/
B、runner 不動
看 *service / deadline*
/ delivery / J

只改期限餘裕 → 改變緊急分支提早量 → 影響期限與送達

URGENT_MARGIN_S (緊急餘裕秒數) | 來源：Lab C marked block | 作用：提供期限分支的比較值
單位：秒 | 判讀：觀察緊急動作、送達、期限與端點 J

Lab B：穩定觀察如何影響動作

這一步用 Lab B 示範品質條件：穩定次數會改變 action 何時出現

① 檔案／區段

student_policy.py
lab-b-enter-exit-hold

② Before (唯一變更)

穩定 2 步
才保持

③ After (唯一變更)

穩定 1 步
即可

④ 固定／觀察

ENTER=2、EXIT=1
； A/C、runner 不動
看 transition / service
/ J

只改穩定步數 → 改變可送模式出現時間 → 影響送達與服務

STABLE_STEPS (穩定步數) | 來源：Lab B marked block | 作用：決定可送模式何時成立
單位：steps | 判讀：比較 transition、送達、服務與端點 J

動作之後要讀哪些事件

這一步把動作接到證據：先看到狀態事件，再判斷服務與成本

① WAIT 後

清醒閒置時間增加
改看 `awake-idle`

② SLEEP 後

休息再喚醒
改看 `sleep / wake`

③ SEND 後

處理、發射、接收
再看 `packet` 是否送達

④ 結果

服務成敗與端點焦耳
`endpoint_energy_j`

動作 → 狀態／封包事件 → 服務結果 → 端點焦耳；不能倒過來

`endpoint_energy_j`（端點能量） | 來源：端點無線與處理模型 | 作用：累積狀態與轉換能量
單位：焦耳 | 判讀：只屬端點證據，不等於系統能量

縮排與回傳決定程式走哪條路

這一步只讀一小段程式：看縮排如何選分支，再把動作交回執行器

① 先看意思

窗口關閉就不送
`if ...: return SLEEP`

② 縮排做什麼

把條件放進正確分支
只修 active block

③ 先檢查

執行 `py_compile`
只檢查語法、不改檔

④ 下一步／看什麼

通過後跑 baseline
讀 result / replay
pair

縮排 → 分支 → 回傳 → runner 事件；語法檢查不改內容

`return`（回傳） | 來源：policy function | 作用：把一個合法動作交給執行器
單位：動作值 | 判讀：語法通過後再看 result / replay 的實際事件

先建立基準結果再比較

這一步先保存沒有改動的基準，下一步才比較候選值

① 固定輸入

packaged policy + 固定情境
不讀未來結果

② 執行命令

只跑 baseline
`bash course.sh run --lab A --case baseline`

③ 產生結果

兩個 JSON
result.json / endpoint-replay.json
原始程式不回寫

④ 下一步／看什麼

確認 status: OK、路徑
先 service，再成本

先保存基準，再凍結規則，最後比較候選；結果不能回頭改 block

`result_path`（結果檔路徑） | 來源：命令輸出 | 作用：定位 result.json
單位：路徑文字 | 判讀：網站只匯入 result.json；endpoint replay 只做配對，runner 不是網站